LEARNING PLAN

Unit Title: APPLY COMPUTER PROGRAMMING PRINCIPLES	**Unit code: IT/CU/ICTA/CC/02/5/MA**
Name of Trainer: Musumbi Denis	**PF/No: 20240208632**
Institution: The Rift Valley National Polytechnic	**Level: 6**
Date of preparation: 05-05-2026	**Date of revision:**
Number of learners: 28	**Class: L6ICT-25M**

Skill or job task

1. Apply computer programming skills
2. Demonstrate structured programming skills
3. Demonstrate object-oriented programming skills

Benchmark or Criteria to be used

1. Apply computer programming skills
    - 1.1 Programming language types are identified as per the user requirements.
    - 1.2 Programming paradigms are applied as per user requirements.
    - 1.3 Program development life cycle is applied as per the work requirements.
    - 1.4 Program design tools are applied as per the user requirements.
    - 1.5 Program writing tools are identified as per the system requirements.

2. Demonstrate structured programming skills
    - 2.1 Identifiers are declared as per program design specification.
    - 2.2 Initialization of variables and constants is performed according to program design specifications.
    - 2.3 User-defined data types are applied as per system requirements.
    - 2.4 Computer program input is created as per program design.
    - 2.5 Data control structures in a program are applied as per program design requirements.
    - 2.6 Data structures in a program are applied as per program design specifications.
    - 2.7 Computer program subroutines are created as per user needs.
    - 2.8 Computer program output is coded as per user requirements.
    - 2.9 Computer program debugging is performed as per work procedures.
    - 2.10 Computer program is compiled as per system requirements.

3. Demonstrate object-oriented programming skills
    3.1 Objects and classes are implemented as per work procedures.
    3.2 Objects methods are declared as per application requirements.
    3.3 Namespaces are applied as per wok procedures.
    3.4 Data abstraction concepts are applied as per work procedures.
    3.5 Object encapsulations are applied as per work procedures.
    3.6 Class templates are implemented as per application requirements.
    3.7 Class inheritance is implemented as per application requirements.
    3.8 Polymorphism is implemented as per application requirements.

Week	**Session No**	**Session Title**	**Specific Learning Outcomes**	**Learning Key Points**	**Trainee Activities**	**Resources & References**	**Learning Checks / Assessments**	**Reflections & Date**
1	1&2	Identification of Programming Languages	By the end of the session, the trainee should be able to: a. Identify various programming language types based on their categories. b. Explain the criteria for selecting programming languages. c. Evaluate programming languages based on user requirements.	OVERVIEW OF PROGRAMMING LANGUAGE CATEGORIES - Procedural languages - Object-oriented languages - Functional languages CRITERIA FOR LANGUAGE SELECTION - Performance needs - Development speed - Platform compatibility USER REQUIREMENTS	- Engage in a Group Discussion to identify and categorize various programming language types. - Participate in Think-Pair-Share to discuss criteria for selecting programming languages based on user requirements. - Conduct a Case Study analysis of scenarios requiring different	- Textbooks: 'Programming Language Concepts' - PowerPoint presentations on programming paradigms - Internet connection for research - Online programming language documentation	Knowledge Checks: 1. Oral questioning on language categories. 2. Short quiz on selection criteria. Attitudes: 1. Curiosity about new technologies. 2. Analytical thinking in evaluation.	

				ANALYSIS - Understanding project scope - Identifying target audience - Defining system constraints	programming language choices. Follow up Activity: 1. Research and present on two new programming languages and their ideal use cases. (10 Marks) Due date:			
1	1&2	Application Programming Paradigms	By the end of the session, the trainee should be able to: a. Explain common programming paradigms such as functional and procedural. b. Differentiate between imperative, object-oriented, and declarative paradigms. c. Select an appropriate programming paradigm based on project requirements.	UNDERSTANDING PROGRAMMING PARADIGMS - Definition of a paradigm - Historical context of paradigm evolution KEY PARADIGMS AND CHARACTERISTICS - Functional: stateless computations - Procedural: sequence of instructions - Object-oriented: objects and classes - Imperative: explicit	- Engage in a Brainstorming session to identify characteristics of different programming paradigms. - Participate in a Jigsaw activity to research and present on specific programming paradigms (e.g., one group on functional, another on OOP). - Conduct a Case Study to choose the most suitable	- Textbooks: 'Concepts of Programming Languages' - Online programming tutorials on paradigms - Computers with IDEs for demonstration - Articles on software architecture	Knowledge Checks: 1. Oral questioning on paradigm differences. 2. Concept mapping of programming paradigms. Attitudes: 1. Adaptability to new concepts. 2. Critical thinking in design choices.	

				control flow PARADIGM SELECTION STRATEGY - Matching paradigm to problem domain - Considering maintainability and scalability - Impact on development team workflow	programming paradigm for a given project scenario. Follow up Activity: 1. Compare and contrast procedural and object-oriented programming paradigms with examples. (15 Marks) Due date:			
2	1&2	Understanding the Program Development Life Cycle	By the end of the session, the trainee should be able to: a. Describe the various stages of the program development life cycle (PDLC). b. Explain best practices for adapting the PDLC to different work requirements. c. Analyze how each stage of the PDLC contributes to successful software development.	STAGES OF PDLC - Requirement gathering - Design - Implementation (Coding) - Testing - Deployment - Maintenance BEST PRACTICES IN PDLC - Agile methodologies - Iterative development	- Engage in a Group Discussion to identify and describe the stages of the Program Development Life Cycle. - Participate in a KWL activity to explore current knowledge, desired knowledge, and learned knowledge about PDLC best practices. - Conduct a Case Study on how different	- Textbooks on Software Engineering - Online articles on SDLC/PDLC models - Project management software examples - Case studies of successful software projects	Knowledge Checks: 1. Whiteboard explanation of PDLC stages. 2. Oral questioning on best practices. Attitudes: 1. Planning and foresight. 2. Thoroughness in process understanding.	

				- User involvement - Documentation ADAPTING PDLC - Project size and complexity - Team structure - Client requirements - Time constraints	PDLC models (e.g., Waterfall, Agile) are applied to various project types. Follow up Activity: 1. Design a simple program's PDLC, detailing each stage and relevant activities. (10 Marks) Due date:			
2	1&2	Application of Program Design Tools - Part 1	By the end of the session, the trainee should be able to: a. Identify various program design tools such as flow charts and pseudocode. b. Explain the purpose and components of flow charts. c. Apply pseudocode to represent basic programming logic.	INTRODUCTION TO DESIGN TOOLS - Importance of design before coding - Categories of design tools FLOW CHART FUNDAMENTALS - Standard symbols (start/end, process, decision, input/output) - Drawing guidelines - Representing sequential logic PSEUDOCODE BASICS	- Engage in an Interactive Lecture with Q&A Sessions to identify different program design tools. - Participate in Demonstrations with Participation to learn how to draw basic flow charts for simple algorithms. - Conduct a Peer Teaching session where	- Textbooks on Programming Logic and Design - Flowcharting software/templates - Pseudocode examples and exercises - Whiteboard and markers	Knowledge Checks: 1. Practical exercise (drawing flowcharts). 2. Pseudocode writing assessment. Attitudes: 1. Precision in design. 2. Logical thinking in problem representation.	

				- Definition and purpose - Keywords and structure (IF, ELSE, WHILE, FOR) - Converting algorithms to pseudocode	trainees explain pseudocode syntax and demonstrate simple logic. Follow up Activity: 1. Create a flowchart and pseudocode for calculating the area of a circle. (15 Marks) Due date:			
3	1&2	Application of Program Design Tools - Part 2	By the end of the session, the trainee should be able to: a. Describe decision tables and decision trees as program design tools. b. Develop algorithms for solving computational problems. c. Select appropriate design tools based on user requirements and project complexity.	ADVANCED DESIGN TOOLS - Decision tables for complex conditions - Decision trees for hierarchical decisions ALGORITHM DEVELOPMENT - Steps for creating efficient algorithms - Analyzing algorithm complexity (brief) TOOL SELECTION STRATEGY - Matching tool to problem type - Considering clarity and maintainability	- Engage in a Think-Pair-Share to discuss the utility of decision tables and decision trees. - Participate in Brainstorming to develop algorithms for common programming tasks. - Conduct Case Studies where trainees select and justify the use of specific design tools for given project scenarios. Follow up	- Textbooks on Discrete Mathematics or Algorithms - Examples of decision tables/trees - Algorithm design textbooks - Project requirement specifications (sample)	Knowledge Checks: 1. Practical exercise (creating decision tables/trees). 2. Algorithm design and explanation. Attitudes: 1. Problem-solving orientation. 2. Efficiency in solution design.	

				- Benefits of integrated tool usage	Activity: 1. Create a decision table for a user login system with multiple authentication methods. (15 Marks) Due date:			
3	1&2	Identification of Program Writing Tools	By the end of the session, the trainee should be able to: a. Identify common program writing tools such as text editors and compilers. b. Describe the functions of linkers, debuggers, and Integrated Development Environments (IDEs). c. Evaluate programming tools based on system requirements and developer preferences.	BASIC PROGRAMMING TOOLS - Text editors (e.g., Notepad++, VS Code) - Purpose of compilers (source code to machine code) ADVANCED DEVELOPMENT TOOLS - Linkers: combining object files - Debuggers: identifying and fixing errors - IDEs: integrated suite of tools (editor, compiler, debugger, etc.) TOOL EVALUATION	- Engage in an Interactive Lecture to identify common program writing tools. - Participate in Demonstrations with Participation to observe the basic functions of a compiler and debugger within an 'IDE'. - Conduct a Group Discussion to evaluate different 'IDEs' based on specified system requirements and developer preferences.	- Computers with various IDEs installed (VS Code, Code::Blocks) - Textbooks on programming environments - Online documentation for IDEs - Articles on developer tools	Knowledge Checks: 1. Tool identification quiz. 2. Functional descriptions of development tools. Attitudes: 1. Resourcefulness in tool selection. 2. Adaptability to different development environments.	

				AND SELECTION - Performance considerations - Features and plugins - Community support - Cost and licensing	Follow up Activity: 1. Research and compare two different IDEs suitable for C/C++ development, listing their pros and cons. (10 Marks) Due date:			
4	1&2	CAT 1 (Continuous Assessment Test 1)	By the end of the session, the trainee should be able to: a. Demonstrate comprehensive understanding of programming language types and paradigms. b. Apply program development life cycle principles. c. Utilize program design and writing tools effectively.	ASSESSMENT COVERAGE - Programming language types - Programming paradigms - Program Development Life Cycle stages TOOLS FOR PROGRAMMING - Program design tools (Flowcharts, Pseudocode, Decision Tables) - Program writing tools (Editors, Compilers, IDEs) EXAM STRATEGY - Time management during the exam	- Engage in a Quiz Game to review key concepts from 'Apply Computer Programming Skills'. - Participate in Q&A Sessions to clarify any doubts regarding the covered topics. - Complete the Continuous Assessment Test (CAT 1) to evaluate understanding of programming principles. Follow up Activity: 1. Review CAT	- CAT 1 question paper - Writing materials - Calculator (if permitted) - Course notes and textbooks	Knowledge Checks: 1. Graded assessment of theoretical knowledge. 2. Evaluation of practical application in problem-solving. Attitudes: 1. Honesty and integrity during assessment. 2. Self-reflection on learning progress.	

				- Understanding question types - Reviewing core concepts	1 feedback and identify areas for improvement. (0 Marks) Due date:			
4	1&2	Identifiers, Variables and Constants in C Language	By the end of the session, the trainee should be able to: a. Declare identifiers in C language following naming conventions and best practices. b. Explain the importance of proper initialization for variables and constants. c. Perform initialization of variables and constants according to program design specifications.	C IDENTIFIERS - Naming rules (alphanumeric, underscore) - Keywords vs. identifiers - Best practices for readability (camelCase, snake_case) VARIABLES AND CONSTANTS - Declaration syntax (data_type variable_name;) - Definition of a constant - Using 'const' keyword INITIALIZATION TECHNIQUES - Direct initialization - Assignment initialization	- Engage in an Interactive Lecture covering identifier rules and variable/constant declaration. - Participate in Demonstrations with Participation on declaring and initializing variables and constants in a C program. - Conduct a Peer Teaching session where trainees demonstrate correct identifier naming and initialization practices. Follow up Activity: 1. Write a C program to	- Computers with C IDE (e.g., Code::Blocks, GCC) - Textbooks on C Programming - C Programming reference guides (online/offline) - Whiteboard and markers	Knowledge Checks: 1. Code review of variable declarations. 2. Q&A on C syntax and initialization. Attitudes: 1. Attention to detail in coding. 2. Adherence to programming best practices.	

				- Importance of preventing garbage values	declare and initialize different types of variables and constants, and print their values. (10 Marks) Due date:			
5	1&2	Applying User-Defined Data Types in C	By the end of the session, the trainee should be able to: a. Explain the concept of user-defined data types in C language. b. Implement structures and enumerations to organize related data. c. Select appropriate user-defined data types based on system requirements.	INTRODUCTION TO USER-DEFINED TYPES - Need for custom data types - Distinction from built-in types STRUCTURES AND ENUMERATIONS - Defining 'struct' (grouping heterogeneous data) - Accessing structure members - Defining 'enum' (named integer constants) DATA TYPE SELECTION - Matching data type to data representation - Memory efficiency considerations	- Engage in an Interactive Lecture to introduce user-defined data types in C. - Participate in Demonstrations with Participation on declaring and using structures and enumerations in C programs. - Conduct a Case Study where trainees design appropriate user-defined data types for a given real-world entity (e.g., a 'Student' or 'Book'). Follow up Activity:	- Computers with C IDE - Textbooks on C Data Structures - C language tutorials (user-defined types) - Examples of complex data organization	Knowledge Checks: 1. Code implementation of structs and enums. 2. Concept Mapping of data type selection criteria. Attitudes: 1. Organization of data. 2. Problem-solving with custom types.	

				- Readability and maintainability	1. Write a C program using a 'struct' to store and display student information (name, ID, age). (15 Marks) Due date:			
5	1&2	Computer Program Input and Output in C	By the end of the session, the trainee should be able to: a. Create computer program input mechanisms using standard C library functions. b. Code computer program output to display information clearly and effectively. c. Integrate input and output operations to build interactive C programs.	STANDARD INPUT FUNCTIONS - 'scanf()' for formatted input - 'getchar()' for single character input - Handling input buffer issues STANDARD OUTPUT FUNCTIONS - 'printf()' for formatted output - 'putchar()' for single character output - Escape sequences for formatting INTERACTIVE PROGRAM DESIGN - Prompting user for input	- Engage in an Interactive Lecture on standard input/output functions in C. - Participate in Demonstrations with Participation on writing C programs that take user input and display formatted output. - Conduct a Round Robin exercise where trainees demonstrate and explain their simple I/O programs. Follow up Activity: 1. Write a C program that	- Computers with C IDE - Textbooks on C Standard I/O - Online C I/O function documentation - Examples of interactive console applications	Knowledge Checks: 1. Practical coding of I/O operations. 2. Code walkthrough for correctness. Attitudes: 1. User-centric design of interfaces. 2. Clarity in program output.	

				- Displaying meaningful results - Error handling for invalid input	prompts the user for two numbers, calculates their sum, and displays the result with a descriptive message. (10 Marks) Due date:			
6	1&2	Data Control Structures in C - Selection & Sequence	By the end of the session, the trainee should be able to: a. Explain the concept of sequential program execution. b. Implement selection control structures (if, else-if, switch) in C programs. c. Apply selection control structures to make decisions within a program flow.	SEQUENTIAL EXECUTION - Programs execute statement by statement - Order of operations - Basic flow of control SELECTION (CONDITIONAL) STATEMENTS - 'if' statement for single condition - 'if-else' for two-way decisions - 'else-if' ladder for multiple conditions - 'switch' statement for multi-way branching PRACTICAL	- Engage in an Interactive Lecture to explain sequential execution and selection control structures. - Participate in Demonstrations with Participation on writing C programs using 'if-else' and 'switch' statements. - Conduct a Think-Pair-Share activity to design logic for a simple calculator using selection statements.	- Computers with C IDE - Textbooks on C Control Structures - Flowchart examples for decision making - Online tutorials for conditional logic	Knowledge Checks: 1. Code implementation of selection structures. 2. Tracing code execution with various inputs. Attitudes: 1. Precision in logic. 2. Logical reasoning in problem-solving.	

				APPLICATION - Using relational and logical operators - Nested 'if' statements - Designing conditional logic	Follow up Activity: 1. Write a C program to determine if a given year is a leap year using selection statements. (15 Marks) Due date:			
6	1&2	Data Control Structures in C - Loops	By the end of the session, the trainee should be able to: a. Describe different types of loop control structures in C (for, while, do-while). b. Implement iteration using 'for', 'while', and 'do-while' loops in C programs. c. Apply looping constructs to perform repetitive tasks efficiently.	INTRODUCTION TO LOOPS - Purpose of iteration - Difference between entry-controlled and exit-controlled loops 'FOR' LOOP - Initialization, condition, increment/decrement - Looping for a fixed number of times 'WHILE' AND 'DO-WHILE' LOOPS - 'while': executes zero or more times - 'do-while': executes one or more times - Choosing the appropriate loop type	- Engage in an Interactive Lecture explaining the different types of loop control structures. - Participate in Demonstrations with Participation on writing C programs that use 'for', 'while', and 'do-while' loops for various tasks. - Conduct a Group Discussion to compare and contrast the usage of 'for', 'while', and 'do-while' loops in different	- Computers with C IDE - Textbooks on C Looping Constructs - Algorithm examples involving iteration - Debugging tools for loop analysis	Knowledge Checks: 1. Code implementation of loops. 2. Debugging loop logic. Attitudes: 1. Efficiency in repetitive tasks. 2. Persistence in debugging.	

					scenarios. Follow up Activity: 1. Write a C program to calculate the factorial of a number using a 'for' loop. (15 Marks) Due date:			
7	1&2	Data Structures in C (Arrays, Queue, Stack, Linked Lists)	By the end of the session, the trainee should be able to: a. Describe common data structures including arrays, queues, and stacks. b. Implement arrays and basic operations on them in C programs. c. Explain the concepts of linked lists and their advantages.	INTRODUCTION TO DATA STRUCTURES - Organizing data for efficient access and manipulation - Static vs. dynamic data structures ARRAYS - One-dimensional arrays - Multi-dimensional arrays - Accessing elements, traversing arrays QUEUES AND STACKS - Queue: FIFO (First-In, First-Out) - Stack: LIFO (Last-In, First-Out)	- Engage in an Interactive Lecture to introduce common data structures. - Participate in Demonstrations with Participation on declaring and manipulating arrays in C. - Conduct a Concept Mapping exercise to visually represent the relationships and differences between arrays, queues, stacks, and linked lists. Follow up Activity:	- Computers with C IDE - Textbooks on Data Structures in C - Visualizations of data structures (online) - Example code for array manipulation	Knowledge Checks: 1. Practical coding (arrays). 2. Diagramming data structures and their operations. Attitudes: 1. Problem-solving with structured data. 2. Organization of information.	

				- Basic operations (enqueue/dequeue, push/pop) LINKED LISTS (CONCEPTUAL) - Nodes, pointers - Advantages over arrays (dynamic size) - Types of linked lists (singly, doubly, circular)	1. Write a C program to implement a simple stack using an array, demonstrating push and pop operations. (20 Marks) Due date:			
7	1&2	Creating C Computer Program Subroutines (Functions)	By the end of the session, the trainee should be able to: a. Explain the benefits of using subroutines (functions) in C programs. b. Design and implement functions to perform specific tasks. c. Utilize functions with parameters and return values to meet user needs.	INTRODUCTION TO FUNCTIONS - Modularity and reusability - Code organization - Simplifying complex problems FUNCTION DEFINITION AND DECLARATION - Function prototype - Function body - Parameters and arguments - Return types FUNCTION CALLS AND SCOPE	- Engage in an Interactive Lecture on the benefits and concepts of C functions. - Participate in Demonstrations with Participation on defining, declaring, and calling functions with various parameters and return types. - Conduct a Peer Teaching session where trainees explain and demonstrate	- Computers with C IDE - Textbooks on C Functions - C programming examples with functions - Online documentation for standard library functions	Knowledge Checks: 1. Code implementation of functions. 2. Explanation of function calls and parameter passing. Attitudes: 1. Modularity in code design. 2. Collaboration through function usage.	

				- Passing arguments by value and reference - Local and global variables - Recursive functions (brief)	a function they have written. Follow up Activity: 1. Write a C program that uses a function to calculate the greatest common divisor (GCD) of two numbers. (15 Marks) Due date:			
8	1&2	Debugging and Compiling C Computer Programs	By the end of the session, the trainee should be able to: a. Perform basic debugging techniques to identify and fix errors in C programs. b. Explain the steps involved in the compilation process of a C program. c. Compile C programs successfully, ensuring compliance with system requirements.	DEBUGGING TECHNIQUES - Syntax errors vs. logical errors - Using a debugger (breakpoints, stepping) - Print statements for tracing SYSTEMATIC DEBUGGING PROCEDURES - Reproducing the bug - Isolating the problem - Fixing and testing COMPILATION PROCESS	- Engage in an Interactive Lecture on debugging techniques and the compilation process. - Participate in Demonstrations with Participation to observe and practice using an 'IDE's' debugger for a sample C program with errors. - Conduct a Crowdsourcing session where trainees share common	- Computers with C IDE and debugger - Textbooks on C Programming (debugging section) - Online debugging tutorials - Sample C programs with known bugs	Knowledge Checks: 1. Practical debugging exercise. 2. Explanation of compilation phases. Attitudes: 1. Problem-solving mindset. 2. Patience and methodical approach.	

				- Preprocessing, compiling, assembling, linking - Role of the compiler and linker - Handling compilation errors and warnings	compilation errors and how they resolved them. Follow up Activity: 1. Debug a provided C program containing logical errors and ensure it compiles and runs correctly. (20 Marks) Due date:			
8	1&2	Implementing Objects and Classes in C++	By the end of the session, the trainee should be able to: a. Explain the fundamental concepts of objects and classes in Object-Oriented Programming ('OOP'). b. Define classes with attributes and behaviors in C++. c. Create objects from classes and demonstrate their instantiation.	INTRODUCTION TO OOP - Paradigm shift from structured programming - Benefits of OOP (reusability, maintainability) CLASSES AS BLUEPRINTS - Definition of a class (data members, member functions) - Access specifiers (public, private, protected) - Constructors and destructors	- Engage in an Interactive Lecture to introduce 'OOP' concepts, objects, and classes. - Participate in Demonstrations with Participation on defining simple classes and creating objects in C++. - Conduct a Case Study on designing a class hierarchy for a real-world entity (e.g., 'Car'	- Computers with C++ IDE (e.g., VS Code, Visual Studio) - Textbooks on C++ OOP - Online C++ tutorials for classes/objects - Diagrams illustrating class-object relationship	Knowledge Checks: 1. Code implementation of classes and objects. 2. Q&A on class/object concepts. Attitudes: 1. Design thinking. 2. Abstraction in modeling.	

				OBJECTS AS INSTANCES - Creating objects from classes - Accessing object members - Memory allocation for objects	or 'Animal'). Follow up Activity: 1. Write a C++ program to define a 'Book' class with attributes like title, author, and price, and create two 'Book' objects. (15 Marks) Due date:			
9	1&2	Declaring Object Methods in C++	By the end of the session, the trainee should be able to: a. Define member functions (methods) within a C++ class. b. Implement methods that perform specific operations on an object's data. c. Apply best practices for naming and designing object methods to fulfill application requirements.	MEMBER FUNCTIONS - Functions associated with a class - Operating on an object's data - Defining inside/outside class METHOD IMPLEMENTATION - Accessing private data members - Parameter passing to methods - Return values from methods BEST PRACTICES	- Engage in an Interactive Lecture on declaring and defining object methods. - Participate in Demonstrations with Participation on adding member functions to existing classes and calling them through objects. - Conduct a Think-Pair-Share to discuss and apply best practices for method naming and	- Computers with C++ IDE - C++ OOP Textbooks (methods section) - Online C++ code examples for member functions - Code style guides	Knowledge Checks: 1. Code implementation of methods. 2. Code review for method design. Attitudes: 1. Modularity in code. 2. Clarity and expressiveness in design.	

				- Clear and descriptive names - Single responsibility principle - Getter and setter methods	functionality. Follow up Activity: 1. Enhance the 'Book' class from the previous exercise by adding methods to set and get book details, and a method to display them. (15 Marks) Due date:			
9	1&2	Applying Namespaces in C++	By the end of the session, the trainee should be able to: a. Explain the purpose and benefits of using namespaces in C++. b. Define and organize code within custom namespaces. c. Apply namespaces to avoid naming conflicts in larger projects.	INTRODUCTION TO NAMESPACES - Solving naming collisions - Organizing code into logical groups - Preventing global scope pollution DECLARING AND USING NAMESPACES - 'namespace' keyword - 'using' directive - Scope resolution operator '::' NESTED AND ANONYMOUS	- Engage in an Interactive Lecture to explain the concept and role of namespaces in C++. - Participate in Demonstrations with Participation on defining and using namespaces in C++ programs. - Conduct a Group Discussion on scenarios where namespaces are essential to manage code	- Computers with C++ IDE - C++ documentation on namespaces - Online tutorials for C++ namespaces - Large codebase examples (conceptual)	Knowledge Checks: 1. Code implementation of namespaces. 2. Explanation of namespace resolution. Attitudes: 1. Organization of code. 2. Preventative design for large projects.	

				NAMESPACES - Structuring namespaces hierarchically - Localizing names within a translation unit	organization. Follow up Activity: 1. Create a C++ program with two functions having the same name but in different namespaces, and demonstrate calling both. (10 Marks) Due date:			
10	1&2	Data Abstraction and Object Encapsulation in C++	By the end of the session, the trainee should be able to: a. Define and explain the concepts of data abstraction and encapsulation in 'OOP'. b. Implement data abstraction by hiding complex details and showing essential features. c. Apply object encapsulation to bundle data and methods, protecting internal state.	DATA ABSTRACTION - Showing only essential information - Hiding implementation details - Using abstract classes/interfaces (briefly) OBJECT ENCAPSULATION - Bundling data and methods into a single unit (class) - Controlling access to data (private, public) - Information hiding	- Engage in an Interactive Lecture to define data abstraction and encapsulation. - Participate in Demonstrations with Participation on implementing classes that use private data members and public methods (encapsulation). - Conduct a Role Play where one trainee acts as the 'user' of an object and another as the 'designer' to	- Computers with C++ IDE - C++ OOP Textbooks - Conceptual diagrams of 'OOP' principles - Articles on software design principles	Knowledge Checks: 1. Code implementation demonstrating encapsulation. 2. Explanation of abstraction principles. Attitudes: 1. Security in software design. 2. Maintainability of code.	

				RELATIONSHIP AND IMPORTANCE - Abstraction achieved through encapsulation - Benefits for security and maintainability - Getter and setter methods as controlled access	demonstrate abstraction. Follow up Activity: 1. Design a C++ class for a 'Bank Account' that uses private members for balance and account number, and public methods for deposit, withdraw, and check balance. (20 Marks) Due date:			
10	1&2	Class Templates Implementation in C++	By the end of the session, the trainee should be able to: a. Explain the concept and benefits of using class templates in C++. b. Implement generic classes using class templates to work with different data types. c. Apply class templates to create reusable and type-safe code.	INTRODUCTION TO TEMPLATES - Generic programming - Writing code once for multiple types - Avoiding code duplication DEFINING CLASS TEMPLATES - 'template <typename T>' syntax - Using type parameter 'T' within the class	- Engage in an Interactive Lecture to introduce the concept of class templates. - Participate in Demonstrations with Participation on defining and using a simple class template (e.g., a generic 'Pair' or 'Stack' class). - Conduct a Peer Teaching session where	- Computers with C++ IDE - C++ OOP Textbooks on templates - Online examples of generic programming - C++ Standard Template Library (STL) overview	Knowledge Checks: 1. Code implementation of class templates. 2. Explanation of template instantiation. Attitudes: 1. Reusability in design. 2. Generality in programming solutions.	

				- Member functions of template classes INSTANTIATING AND USING TEMPLATES - Creating objects of template classes with specific types - Benefits for data structures (e.g., generic Stack, Queue)	trainees demonstrate how to use a template class they implemented. Follow up Activity: 1. Write a C++ class template for a generic 'Array' that can store elements of any data type and includes methods for adding and accessing elements. (20 Marks) Due date:			
11	1&2	Class Inheritance Implementation in C++ - Part 1	By the end of the session, the trainee should be able to: a. Define the concept of inheritance and its importance in 'OOP'. b. Implement single inheritance to create derived classes from base classes. c. Explain different types of inheritance relationships (public, private,	INTRODUCTION TO INHERITANCE - Code reusability - Establishing 'is-a' relationships - Hierarchy of classes BASE AND DERIVED CLASSES - Syntax for inheritance ('class Derived : access Base')	- Engage in an Interactive Lecture to introduce inheritance concepts and terminology. - Participate in Demonstrations with Participation on creating a simple base class and a derived class using public inheritance in	- Computers with C++ IDE - C++ OOP Textbooks on Inheritance - Diagrams of class hierarchies - Online tutorials for C++ inheritance	Knowledge Checks: 1. Code implementation of single inheritance. 2. Explanation of inheritance types and access rules. Attitudes: 1. Hierarchical thinking in design. 2. Reusability of code components.	

			protected).	- Constructor/destructor call order - Overriding base class methods ACCESS SPECIFIERS IN INHERITANCE - Public inheritance: public -> public, protected -> protected - Protected inheritance - Private inheritance	C++. - Conduct a Case Study to design a class hierarchy for a set of related objects (e.g., 'Vehicle' -> 'Car', 'Bike'). Follow up Activity: 1. Create a base class 'Shape' with a method 'calculateArea()' and a derived class 'Circle' that inherits from 'Shape' and overrides 'calculateArea()' . (20 Marks) Due date:			
11	1&2	Class Inheritance Implementation in C++ - Part 2	By the end of the session, the trainee should be able to: a. Implement multiple and multilevel inheritance scenarios in C++. b. Describe hierarchical inheritance and its applications. c. Utilize virtual functions to enable runtime	ADVANCED INHERITANCE TYPES - Multilevel inheritance (A->B->C) - Multiple inheritance (A, B -> C) - Hierarchical inheritance (A -> B, A -> C) AMBIGUITY IN	- Engage in an Interactive Lecture explaining advanced inheritance types and virtual functions. - Participate in Demonstrations with Participation on implementing	- Computers with C++ IDE - C++ OOP Textbooks on advanced inheritance - Complex class hierarchy examples - Articles on design patterns using inheritance	Knowledge Checks: 1. Code implementation of multilevel inheritance. 2. Discussion of inheritance complexities. Attitudes: 1. Complex problem-solving. 2. Structural thinking in	

			polymorphism (preparation for next session).	MULTIPLE INHERITANCE - Diamond problem - Virtual base classes (brief) VIRTUAL FUNCTIONS - Preparing for polymorphism - Runtime binding - Pointers to base class	multilevel inheritance and setting up virtual functions in C++. - Conduct a Group Discussion on the advantages and challenges of different inheritance types. Follow up Activity: 1. Extend the 'Shape' and 'Circle' example by adding another derived class 'Rectangle' and demonstrating hierarchical inheritance with common methods. (15 Marks) Due date:		class design.	
12	1&2	Implementing Class Polymorphism in C++	By the end of the session, the trainee should be able to: a. Define the concept of polymorphism and its significance in 'OOP'. b. Implement	INTRODUCTION TO POLYMORPHISM - 'Many forms' - Ability of an object to take on many forms	- Engage in an Interactive Lecture to explain polymorphism and its importance. - Participate in	- Computers with C++ IDE - C++ OOP Textbooks on Polymorphism - Real-world examples of	Knowledge Checks: 1. Code implementation of polymorphism. 2. Explanation of virtual function call	

			runtime polymorphism using virtual functions and pointers/references. c. Apply polymorphism to achieve flexible and extensible code design.	- Runtime vs. Compile-time polymorphism (briefly) RUNTIME POLYMORPHISM (DYNAMIC BINDING) - Virtual functions in base class - Pointers/references to base class objects - Function overriding PRACTICAL APPLICATION - Designing flexible interfaces - Achieving extensibility in systems - Example: drawing different shapes via a base pointer	Demonstrations with Participation on implementing polymorphism using virtual functions in C++. - Conduct a Case Study on how polymorphism is used in real-world software design (e.g., GUI event handling). Follow up Activity: 1. Modify the 'Shape' hierarchy to demonstrate polymorphism, where a function can draw different shapes ('Circle', 'Rectangle') using a 'Shape' pointer. (25 Marks) Due date:	polymorphic behavior - Articles on design patterns (Strategy, Factory)	mechanism. Attitudes: 1. Flexibility in system design. 2. Extensibility of software solutions.	
12	1&2	CAT 3 (Final Continuous Assessment Test)	By the end of the session, the trainee should be able to: a. Demonstrate proficiency in	COMPREHENSIVE ASSESSMENT - Structured Programming	- Engage in a Quiz Game to review key concepts across structured and	- CAT 3 question paper - Computers for practical section	Knowledge Checks: 1. Graded assessment of theoretical and	

			structured programming skills. b. Apply object-oriented programming principles effectively. c. Integrate various programming concepts to solve complex problems.	(identifiers, data types, control structures, functions) - Object-Oriented Programming (classes, objects, inheritance, polymorphism) - Problem-solving and debugging EXAM SCOPE - Review of all unit learning outcomes - Practical application of C and C++ programming concepts - Theoretical understanding of principles TEST READINESS - Reviewing course materials - Practicing coding exercises - Time management for the exam	object-oriented programming. - Participate in Q&A Sessions to address any last-minute questions before the final assessment. - Complete the Final Continuous Assessment Test (CAT 3) to demonstrate overall competency in applying computer programming principles. Follow up Activity: 1. Reflect on learning journey and identify areas for continuous improvement in programming skills. (0 Marks) Due date:	- Writing materials - All course textbooks and notes	practical programming skills. 2. Evaluation of problem-solving ability through coding challenges. Attitudes: 1. Professionalism in problem-solving. 2. Commitment to lifelong learning in programming.	